



UD7_Práctica3 - Cursores.

Índice

UD7_Práctica3 - Cursores.....	1
Documentación e información.....	2
O que hai que entregar.....	2
Cursores.....	3
Exercicios sobre cursores.....	3



Documentación e información.

Tedes que crear as BBDD indicadas en cada exercicio. Os datos para a táboa a crear están incluídos no enunciado da tarefa.

Moitos dos exercicios fanse sobre as mesmas BD, polo que podedes aproveitar o que teñades feito duns exercicios para outros.

O que hai que entregar.

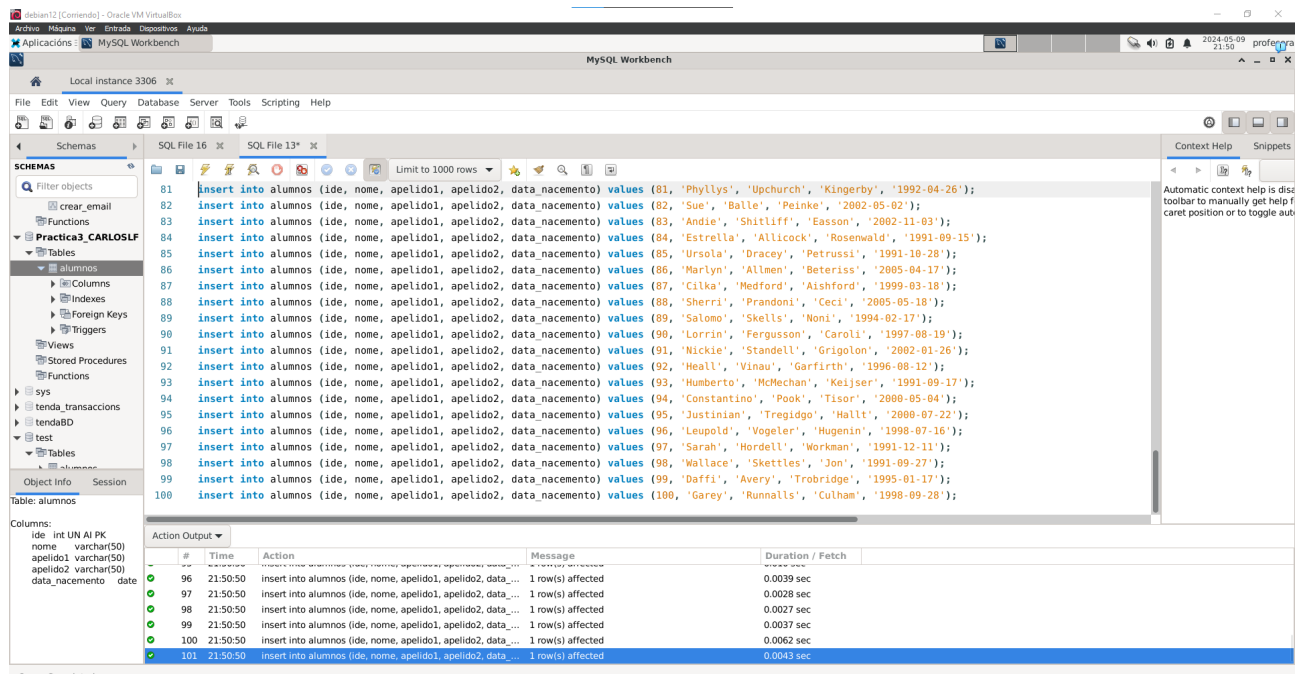
Un documento PDF que inclúa as imaxes seguintes:

- **O código de cada exercicio.** En caso de que o código non quepa nunha imaxe:
 - Copia o código no documento.
 - Inclúe unha imaxe tamén onde se vexa a rutina incluída na BD. Debe verse o nome e a cabeceira da rutina.
- **Os casos de proba** de cada exercicio. Isto inclúe unha imaxe por cada posible resultado da rutina.

As imaxes deben ser válidas: *incluír o teu nome nalgures, o comando, o resultado da execución e o Action Output*



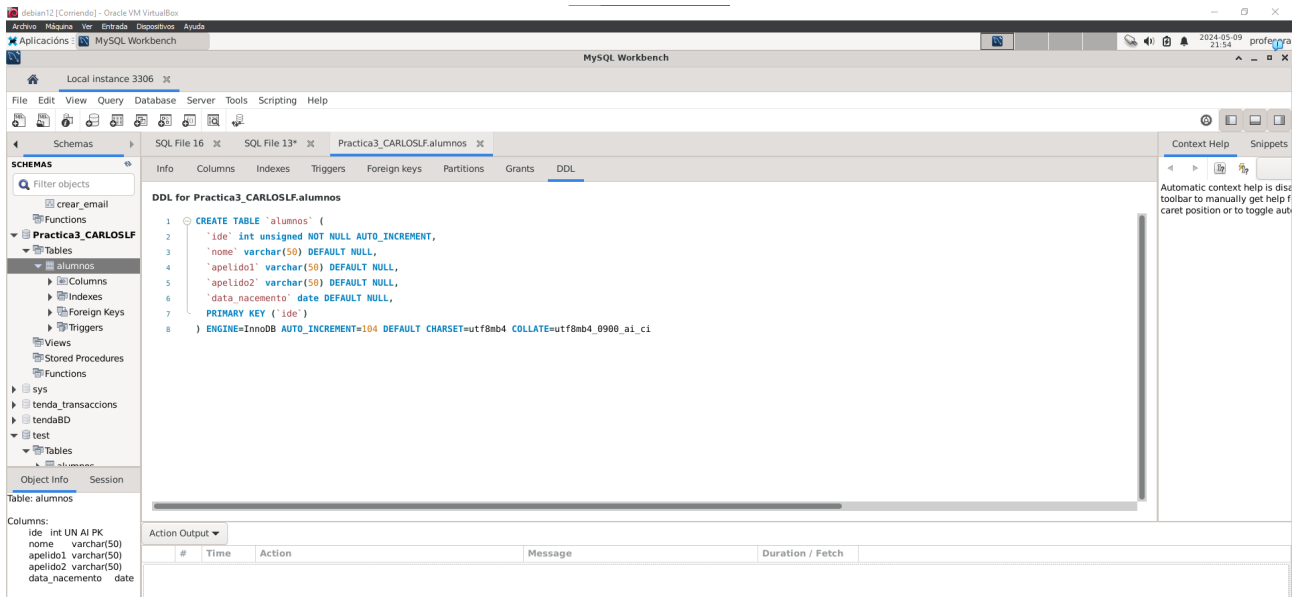
Cursores.



Exercicios sobre cursores.

Ola Mai, esta mensaxe é para dicir que primeiro fixen a “práctica 4” e por iso hay algunhas capturas desde o DDL e non sae o output, espero que te sirvan igual.

1. Escribe as sentenzas SQL necesarias para **crear unha base de datos chamada test**, unha **táboa chamada alumnos** e **4 sentenzas de inserción** para inicializar a táboa. A táboa alumnos está formada polas seguintes columnas:
 - ide (enteiro sen signo e clave primaria)
 - nome (cadea de caracteres)
 - apelido1 (cadea de caracteres)
 - apelido2 (cadea de caracteres)
 - data_nacemento (data)



2. Unha vez creada a táboa decídese **engadir unha nova columna á táboa** chamada **idade** que será un **valor calculado** a partir da columna **data_nacemento**. Escriba a sentenza SQL necesaria para modificar a táboa e engadir a nova columna.
 - Escriba unha **función chamada calcular_idade** que reciba unha data e devolva o número de anos que pasaron desde a data actual ata a data pasada como parámetro:
 - Función: `calcular_idade`
 - Entrada: Data
 - Saída: Número de anos (enteiro)



The screenshot shows the MySQL Workbench interface. The 'Schemas' pane on the left shows the 'Practica3_CARLOSFL' database selected. The 'SQL File 16*' tab contains the following SQL code:

```
1 ALTER TABLE alumnos
2 ADD COLUMN idade INT;
3
4 -- CARLOSFL
```

The 'Action Output' pane at the bottom shows the execution results:

#	Time	Action	Message	Duration / Fetch
1	20:50:02	ALTER TABLE alumnos ADD COLUMN idade INT	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.039 sec

The screenshot shows the MySQL Workbench interface. The 'Schemas' pane on the left shows the 'Practica3_CARLOSFL' database selected. The 'SQL File 4*' tab contains the following SQL code:

```
1 DELIMITER //
2
3 CREATE FUNCTION calcular_idade(fecha_nacimiento DATE)
4 RETURNS INT DETERMINISTIC
5 BEGIN
6 DECLARE edad INT;
7 DECLARE fecha_actual DATE;
8
9 SET fecha_actual = CURDATE();
10 SET edad = YEAR(fecha_actual) - YEAR(fecha_nacimiento);
11
12 IF MONTH(fecha_actual) < MONTH(fecha_nacimiento) OR
13 (MONTH(fecha_actual) = MONTH(fecha_nacimiento) AND DAY(fecha_actual) < DAY(fecha_nacimiento)) THEN
14 SET edad = edad - 1;
15 END IF;
16
17 RETURN edad;
18
19 END //
20 DELIMITER ;
21
22 -- CARLOSFL
```

The 'Action Output' pane at the bottom shows the execution results:

#	Time	Action	Message	Duration / Fetch
1	20:51:52	CREATE FUNCTION calcular_idade(fecha_nacimiento DAT...	0 row(s) affected	0.0083 sec

Éxito:



MySQL Workbench interface showing a query execution. The query is: `select Practica3_CARLOSFL.calcular_idade('2000-05-13');`. The result grid shows one row with the value 24. The function object info shows parameters: `fecha_nacimiento: DATE` and Returns: `int`.

Fallo:

MySQL Workbench interface showing a query execution. The query is: `select Practica3_CARLOSFL.calcular_idade('13-05-2024');`. The error message is: `Error Code: 1292, Incorrect date value: '13-05-2024'`. The function object info shows parameters: `fecha_nacimiento: DATE` and Returns: `int`.

3. Agora escriba un **procedemento** que permita calcular a idade de **todos os alumnos** que xa existen na táboa.

- Para isto será necesario crear un **procedemento** chamado **actualizar_columna_idade** que calcule a idade de cada alumno e actualice a táboa.



- Este procedemento **fará uso da función calcular_idade** que creamos no paso anterior.

The screenshot shows the MySQL Workbench interface. The 'Schemas' pane on the left shows the 'Practica3_CARLOSIF' database. The 'Stored Procedures' pane shows the 'actualizar_columna_idade' procedure. The main editor shows the SQL code for the procedure, which is a cursor-based loop that updates the 'edad' column in the 'alumnos' table. The 'Action Output' pane at the bottom shows the execution results.

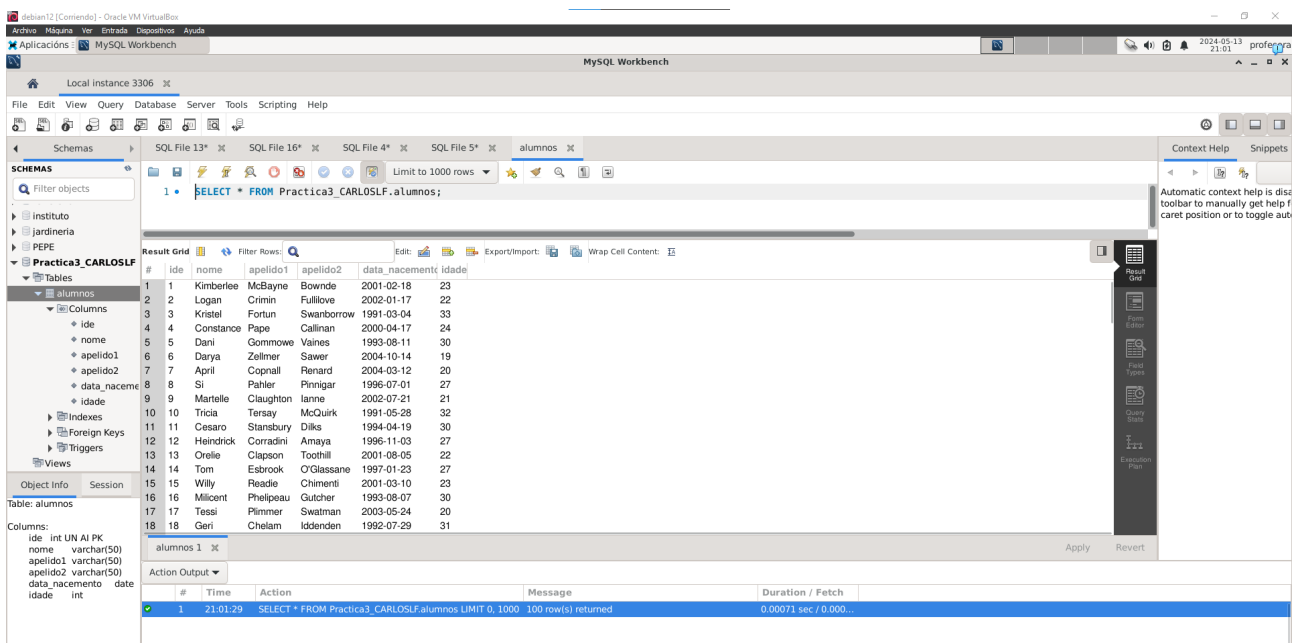
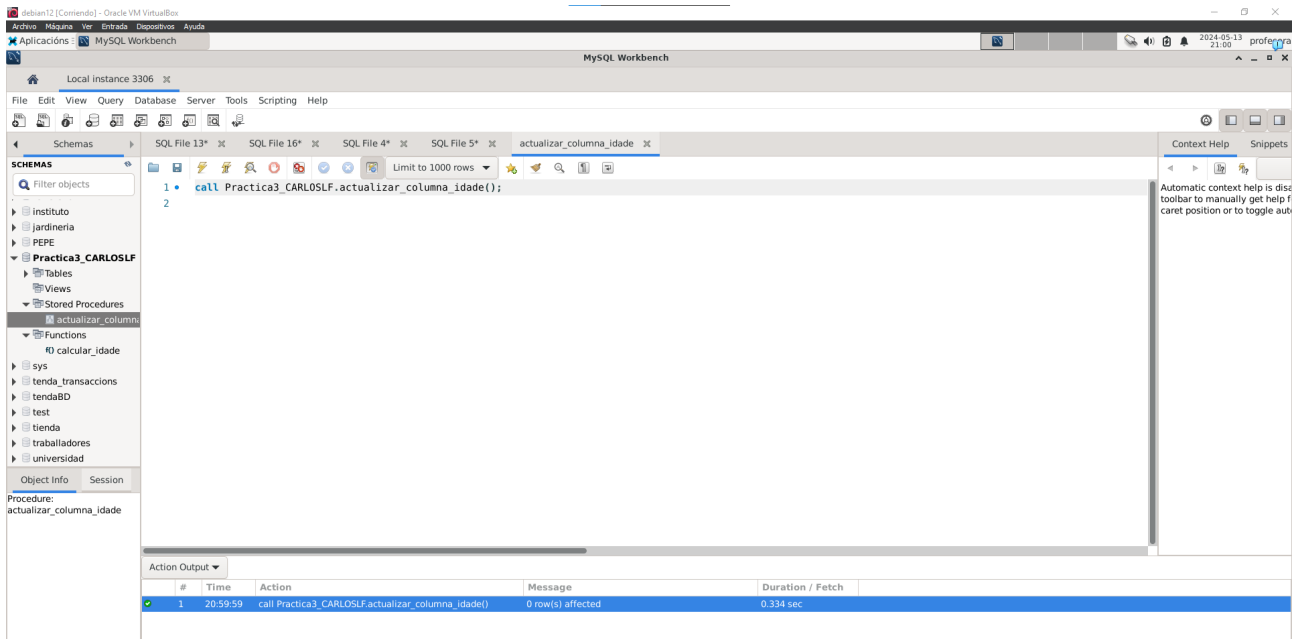
```
1 DELIMITER //
2
3 CREATE PROCEDURE actualizar_columna_idade()
4 BEGIN
5     DECLARE done INT DEFAULT FALSE;
6     DECLARE alumno_id INT;
7     DECLARE fecha_nacimiento DATE;
8     DECLARE edad_alumno INT;
9
10    -- Cursor para recorrer todos los alumnos
11    DECLARE cur CURSOR FOR SELECT id, fecha_nacimiento FROM alumnos;
12    DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = TRUE;
13
14    -- Abrir el cursor
15    OPEN cur;
16
17    -- Iniciar el bucle
18    read_loop: LOOP
19        -- Leer datos del cursor en las variables
20        FETCH cur INTO alumno_id, fecha_nacimiento;
21
22        -- Comprobar si se han acabado los datos
23        IF done THEN
24            LEAVE read_loop;
25        END IF;
26    END LOOP;
```

#	Time	Action	Message	Duration / Fetch
1	20:58:59	CREATE PROCEDURE actualizar_columna_idade() BEGIN ...	0 row(s) affected	0.0070 sec

The screenshot shows the MySQL Workbench interface. The 'Schemas' pane on the left shows the 'Practica3_CARLOSIF' database. The 'Stored Procedures' pane shows the 'actualizar_columna_idade' procedure. The main editor shows the SQL code for the procedure, which is a cursor-based loop that updates the 'edad' column in the 'alumnos' table. The 'Action Output' pane at the bottom shows the execution results.

```
15 OPEN cur;
16
17 -- Iniciar el bucle
18 read_loop: LOOP
19     -- Leer datos del cursor en las variables
20     FETCH cur INTO alumno_id, fecha_nacimiento;
21
22     -- Comprobar si se han acabado los datos
23     IF done THEN
24         LEAVE read_loop;
25     END IF;
26
27     -- Calcular la edad del alumno utilizando la función calcular_idade
28     SET edad_alumno = calcular_idade(fecha_nacimiento);
29
30     -- Actualizar la columna edad en la tabla alumnos
31     UPDATE alumnos SET edad = edad_alumno WHERE id = alumno_id;
32 END LOOP;
33
34 -- Cerrar el cursor
35 CLOSE cur;
36 END//
37
38 DELIMITER ;
39
40 -- CARLOSIF
```

#	Time	Action	Message	Duration / Fetch
1	20:58:59	CREATE PROCEDURE actualizar_columna_idade() BEGIN ...	0 row(s) affected	0.0070 sec



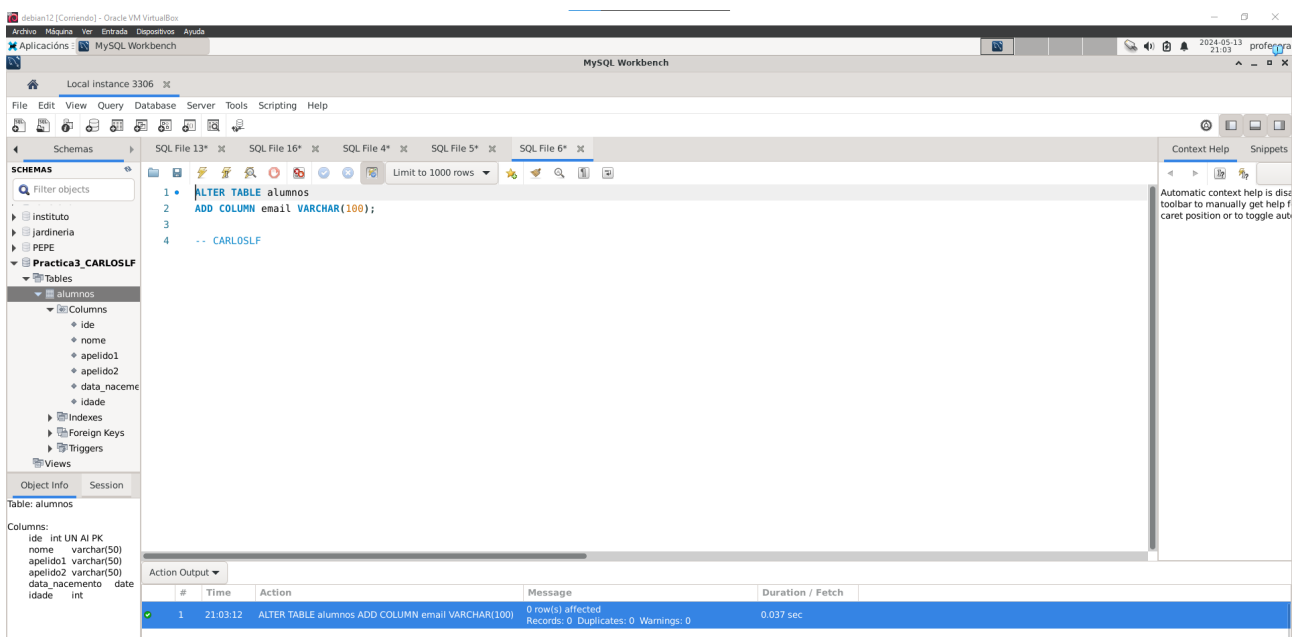
4. Modifica a táboa alumnos do exercicio anterior para **engadir unha nova columna email**. Unha vez que modificamos a táboa necesitamos asignarlle unha dirección de correo electrónico de forma automática.
1. Escriba unha **función chamado crear_email** que dados os parámetros de entrada: nome, apelido1, apelido2 e dominio, cree unha dirección de email e devólva como saída.



- Procedemento: crear_email
- Entrada:
 - nome (cadea de caracteres)
 - apelido1 (cadea de caracteres)
 - apelido2 (cadea de caracteres)
 - dominio (cadea de caracteres)
- Saída: email (cadea de caracteres)

2. Devolva unha dirección de correo electrónico co seguinte formato:

- O primeiro carácter do parámetro nomee.
- Os tres primeiros caracteres do parámetro apelido1.
- Os tres primeiros caracteres do parámetro apelido2.
- O carácter @.
- O dominio pasado como parámetro chamado 'teunome.gal'.





```
DELIMITER //
CREATE FUNCTION crear_email(nombre VARCHAR(50), apellido1 VARCHAR(50), apellido2 VARCHAR(50), dominio VARCHAR(50))
RETURNS VARCHAR(100) DETERMINISTIC
BEGIN
    DECLARE email_resultado VARCHAR(100);
    SET email_resultado = CONCAT(
        LEFT(nombre, 1),
        LEFT(apellido1, 3),
        LEFT(apellido2, 3),
        '0',
        dominio
    );
    RETURN email_resultado;
END//
DELIMITER ;
```

#	Time	Action	Message	Duration / Fetch
1	21:04:57	CREATE FUNCTION crear_email(nombre VARCHAR(50), ap...	0 rows(s) affected	0.0079 sec

```
select Practica3_CARLOSFL.crear_email('Kimberlee', 'McBayne', 'Bownde', 'CARLOSFL.gal');
```

#	Time	Action	Message	Duration / Fetch
1	21:09:06	select Practica3_CARLOSFL.crear_email('Kimberlee', 'McB...	1 row(s) returned	0.00053 sec / 0.000...

- Agora escriba un **procedemento que permita crear un email para todos os alumnos** que xa existen na táboa.
 - Crear un procedemento chamado **actualizar_columna_email** que actualice a columna email de alumnos. Este procedemento fará uso do procedemento crear_email que creamos no paso anterior.



- Escribe un **procedemento chamado crear_lista_emails_alumnos** que devolva a lista de emails da táboa alumnos separados por un punto e coma. Exemplo:
juan@teunome.gal;maria@teunome.gal;pepe@teunome.gal;lucia@teunome.gal

```
1 DELIMITER //
2
3 CREATE PROCEDURE actualizar_columna_email()
4 BEGIN
5     DECLARE done INT DEFAULT FALSE;
6     DECLARE alumno_id INT;
7     DECLARE nombre_alumno VARCHAR(50);
8     DECLARE apellidos_alumno VARCHAR(50);
9     DECLARE apellidos2_alumno VARCHAR(50);
10    DECLARE email_alumno VARCHAR(100);
11
12    -- Cursor para recorrer todos los alumnos
13    DECLARE cur CURSOR FOR SELECT ide, nome, apellidos1, apellidos2 FROM alumnos;
14    DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = TRUE;
15
16    -- Abrir el cursor
17    OPEN cur;
18
19    -- Iniciar el bucle
20    read_loop: LOOP
21        -- Leer datos del cursor en las variables
22        FETCH cur INTO alumno_id, nombre_alumno, apellidos1_alumno, apellidos2_alumno;
23
24        -- Comprobar si se han acabado los datos
25        IF done THEN
26            LEAVE read_loop;
27        END IF;
28    END LOOP;
29
30    -- Cerrar el cursor
31    CLOSE cur;
32
33    DELIMITER ;
34
35 -- CARLOS LF
```

#	Time	Action	Message	Duration / Fetch
1	21:14:13	CREATE PROCEDURE actualizar_columna_email()	0 rows(s) affected	0.0051 sec

```
17 OPEN cur;
18
19 -- Iniciar el bucle
20 read_loop: LOOP
21     -- Leer datos del cursor en las variables
22     FETCH cur INTO alumno_id, nombre_alumno, apellidos1_alumno, apellidos2_alumno;
23
24     -- Comprobar si se han acabado los datos
25     IF done THEN
26         LEAVE read_loop;
27     END IF;
28 END LOOP;
29
30 -- Generar el email del alumno utilizando el procedimiento crear_email
31 SET email_alumno = crear_email(nombre_alumno, apellidos1_alumno, apellidos2_alumno, "teunome.gal");
32
33 -- Actualizar la columna email en la tabla alumnos
34 UPDATE alumnos SET email = email_alumno WHERE ide = alumno_id;
35
36 -- Cerrar el cursor
37 CLOSE cur;
38
39 END//
40
41 DELIMITER ;
42
43 -- CARLOS LF
```

#	Time	Action	Message	Duration / Fetch
1	21:14:13	CREATE PROCEDURE actualizar_columna_email()	0 rows(s) affected	0.0051 sec



MySQL Workbench interface showing the execution of a stored procedure. The left sidebar shows the database structure for 'Practica3_CARLOSFL'. The main window displays the SQL query: `CALL Practica3_CARLOSFL.actualizar_columna_email();`. The bottom panel shows the execution results:

#	Time	Action	Message	Duration / Fetch
1	21:16:27	call Practica3_CARLOSFL.actualizar_columna_email()	0 row(s) affected	0.274 sec

MySQL Workbench interface showing the execution of a SELECT query. The left sidebar shows the database structure for 'Practica3_CARLOSFL'. The main window displays the SQL query: `SELECT * FROM Practica3_CARLOSFL.alumnos;`. The bottom panel shows the execution results in a grid format:

#	ide	nome	apeliido1	apeliido2	data_nacemento	idade	email
1	1	Kimberlee	McBayne	Bownde	2001-02-18	23	KMcBBow@teunome.gal
2	2	Logan	Crimin	Fullove	2002-01-17	22	LCrFu@teunome.gal
3	3	Kristal	Fortun	Swarborow	1991-03-04	33	KFrSw@teunome.gal
4	4	Constance	Pape	Callinan	2000-04-17	24	CPapCa@teunome.gal
5	5	Dani	Gommowe	Vaines	1993-08-11	30	DGomVal@teunome.gal
6	6	Darya	Zellmer	Sawer	2004-10-14	19	DZelSaw@teunome.gal
7	7	April	Copnall	Renard	2004-03-12	20	ACopRen@teunome.gal
8	8	Si	Pahler	Pinnigar	1996-07-01	27	SPahPin@teunome.gal
9	9	Martelle	Claughton	lanne	2002-07-21	21	MClaTan@teunome.gal
10	10	Tricia	Tersay	McQuirk	1991-05-28	32	TTerMcQ@teunome.gal
11	11	Cesaro	Stansbury	Dike	1994-04-19	30	CSaDi@teunome.gal
12	12	Heindrick	Corradini	Amaya	1996-11-03	27	HCoraMa@teunome.gal
13	13	Orelie	Clapson	Toothill	2001-08-05	22	OClaToo@teunome.gal
14	14	Tom	Esbrook	O'Glassane	1997-01-23	27	TEsbO'G@teunome.gal
15	15	Willy	Readie	Chimentli	2001-03-10	23	WRReaChi@teunome.gal
16	16	Milcent	Phelipeau	Gutcher	1993-08-07	30	MPheGut@teunome.gal
17	17	Tessi	Plimmer	Swatman	2003-05-24	20	TPisWa@teunome.gal
18	18	Geni	Chelam	Iddenden	1992-07-29	31	GCHeId@teunome.gal
19	19	Waylan	Wykey	Risby	1991-06-22	32	WWyKRis@teunome.gal
20	20	Huberto	Southwick	Baudy	2002-01-29	22	HSouBau@teunome.gal
21	21	Phyllis	O'Kennedy	Tomas	1996-04-25	28	PO'KTom@teunome.gal



The screenshot shows the MySQL Workbench interface. The 'Schemas' pane on the left shows the 'Practica3_CARLOSFL' database. The main editor shows the following SQL code:

```
1 DELIMITER //
2
3 CREATE PROCEDURE crear_lista_emails_alumnos()
4 BEGIN
5     DECLARE lista_emails TEXT DEFAULT '';
6     DECLARE done INT DEFAULT FALSE;
7     DECLARE email_alumno VARCHAR(100);
8
9     -- Cursor para recorrer todos los alumnos
10    DECLARE cur CURSOR FOR SELECT email FROM alumnos;
11    DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = TRUE;
12
13    -- Abrir el cursor
14    OPEN cur;
15
16    -- Iniciar el bucle
17    read_loop: LOOP
18        -- Leer datos del cursor en la variable
19        FETCH cur INTO email_alumno;
20
21        -- Comprobar si se han acabado los datos
22        IF done THEN
23            LEAVE read_loop;
24        END IF;
25
```

The 'Action Output' pane at the bottom shows the execution results:

#	Time	Action	Message	Duration / Fetch
1	21:31:47	CREATE PROCEDURE crear_lista_emails_alumnos() BEGIN...	0 row(s) affected	0.0051 sec

The screenshot shows the MySQL Workbench interface. The 'Schemas' pane on the left shows the 'Practica3_CARLOSFL' database. The main editor shows the following SQL code:

```
15
16 -- Iniciar el bucle
17 read_loop: LOOP
18     -- Leer datos del cursor en la variable
19     FETCH cur INTO email_alumno;
20
21     -- Comprobar si se han acabado los datos
22     IF done THEN
23         LEAVE read_loop;
24     END IF;
25
26     -- Agregar el email del alumno a la lista
27     SET lista_emails = CONCAT(lista_emails, email_alumno, ',');
28 END LOOP;
29
30 -- Cerrar el cursor
31 CLOSE cur;
32
33 -- Devolver la lista de emails
34 SELECT lista_emails AS Lista_Emails;
35 END//
36
37 DELIMITER ;
38
39 -- CARLOSFL
```

The 'Action Output' pane at the bottom shows the execution results:

#	Time	Action	Message	Duration / Fetch
1	21:31:47	CREATE PROCEDURE crear_lista_emails_alumnos() BEGIN...	0 row(s) affected	0.0051 sec



MySQL Workbench interface showing a query execution result.

Query:

```
call Practica3_CARLOSFL.crear_lista_emails_alumnos();
```

Result Grid:

#	Lista Emails
1	KMcBBow@teunome.gal;LCrFu@teunome.gal;KForSwa@teunome.gal;CPapCal@teunome.gal;DGonVai@teunome.gal;DZelSaw@teunome.gal;A...

Action Output:

#	Time	Action	Message	Duration / Fetch
1	21:32:27	call Practica3_CARLOSFL.crear_lista_emails_alumnos()	1 row(s) returned	0.0032 sec / 0.0000...